Name: Mohamad Khairul Bin Mansor

Future of AI: Model Context Protocol

Introduction

Large Language Model (LLM) is capable of generating contents and giving reasonings for user query based on large amount of pre-trained data. However, LLM suffers from a limitation to connect with the world outside of its training data. Without a capability to access external databases, files and software, it can be considered as an isolated intelligent model. LLM cannot see specific context from the user that is essentially required for making accurate answer. The gap identified can be solved now with Model Context Protocol (MCP).

MCP was designed by Anthropic in order to solve the problem with LLM's context. MCM defines a universal and open standard for any AI application or system to communicate with any external tool or data source using a single interface. MCP has a similar analogy with UBS-C as a universal connector for user device and gadget. The goal of MCP is to solve problem with regards to AI application interoperability instead of creating huge technical and engineering burden.

MCP is particularly special because of how fast the speed and the extensiveness of its application. Within a year of its release as an open-source protocol, MCP had been adapted by other big tech companies such as OpenAI and Google. Eventually, MCP was donated to Agentic AI Foundation under the Linux Foundation and it remains a critical and neutral protocol from any specific AI provider.

Background & Problem Setting

As more LLMs being used in live production environment, a gap emerged between their reasoning layers and their limited contextual awareness. A LLM trained on enormous amount of corpus training data can explain the concept and principles of a term greatly, but it failed to retrieve user context due to no standardised mechanism to connect between the model and user context. For every connection to occur, custom engineering is required which consist of the process writing specific codes by software developer team. This process entails creating codes to call an API from external source, parse its content and present the output to LLM in a format that LLM can understand.

This leads to a problem called the MxN integration problem. If an organisation planned to connect M different type of AI applications to N different type of data tools, the organisation needs to build and maintain MxN distinct integrations. The developers need to create hundreds of custom connectors, each with its own logic and architecture. As the AI ecosystem grow, the combination number of custom connections needed made the entire plan impossible.

Prior to MCP, several approaches were created to tackle this limitation. OpenAI introduced function calling for LLM to allow software developers to define a set of callable functions. With these function tools, the LLM could generate structured JSON payloads specifying which function to call including their parameters. However, this function calling capability could only work with GPT models and required code modification to be compatible with other vendor models. Also, there was no fixed standard on how tool's response should be returned and how LLM and tools communicate with each other.

MCP was created to solve this issue without locking into any specific provider of LLM. Instead of specifying how a certain LLM calls a certain tool, MCP uses a universal protocol between its components. There are 3 components in MCP architecture, which are the host, the client and the server. The host is the AI application or system that control the overall interaction. An example of MCP host is Integrated Development Environment (IDE) and custom AI agent. The client serves as a lightweight connector and maintain one-to-one session between the host with an MCP server. The client converts natural language request from LLM into structured JSON-RCP and translates the response back to the AI system. Lastly, MCP server provides specific data or capabilities such as databases, filesystems and API calls.

The server can be divided into three main categories, which are resources, tools and prompts. The resources give read-only access to data such as file contents and databases. The tools are callable functions that allow AI system to perform certain actions designed such as executing a query. Finally, the prompts are reuseable prompt templates that guide LLM behaviour and interaction. With these three main primitives, they give breadth to the protocol that were lacked in earlier approach of typical function calling. MCP provides complete framework for bidirectional stateful connection.

Analysis & Discussion

Due to MCP strategic capabilities, it gained big support and traction after its official release to the general public as an open-sourced protocol. Alternative options of AI interoperability such as OpenAI's function calling and vendor-specific tool failed to become the universal standard. On the other hand, MCP solved this issue in comparison to other earlier attempts. The technical information and implementation guidelines regarding MCP are released and documented in its official website. The commitment of Anthropic to make MCP a vendor-neutral protocol and a shared infrastructure is commendable. Anthropic eventually made the decision to donate MCP to Linux Foundation, reinforcing Anthropic's policy and commitment for greater good.

Multiple adoptions followed by in the big AI tech companies such as OpenAI, Microsoft and Google. Once these major LLM providers applied MCP in their ecosystem, the unfavourable impact of not following the trend could be observed. For example, a software developer building a new AI application but refusing to apply MCP in the AI architecture is missing huge benefits especially when the developer is utilizing LLM application from these main providers. The perfect timing of MCP release to tackle the problem made the adoption much more dynamic and top-rated.

The implications of MCP to the individuals particularly software developers, IT analysts and IT workers represent a change in how AI assistants comprehend and take in user context. Previously, AI application was restricted in its ability to access specific context from a user. For examples, the user context from the latest version of codes, recent emails or bills and current database record. The users had to manually copy and paste relevant context into their prompts

in order to pass the information needed for inference. With MCP, context which is typically self-contained can be easily transferable to the host. AI assistant configured to MCP servers are seamlessly connected to user's information, data record and file system, thus allowing better LLM reasoning without the need for constant curation of prompts.

Several aspects of MCP deserved careful attention by understanding the associated risks of MCP. Attack on MCP's architecture particularly prompt injection become more dangerous when AI application is configured with MCP and given full access to sensitive information. An individual who using MCP to connect AI system with their file system and personal record is technically providing AI agent full permission over their digital environment. According to MCP's official specification, certain security principles can't be enforced at protocol layer and must be implemented by the individuals. For individuals that are less technical, this security risk is invisible until something goes wrong with their data system.

MCP provides the most immediate benefits to organisations as there is now lower integration engineering overhead needed to complete the AI projects. The MxN problem discussed earlier consumed a lot of engineering resources in any organisation that attempts to scale in building AI agents and applications. With MCP, it is essentially just a M+N problem where client interface is implemented once per AI application. Similarly, server interface is implemented once per data resource or record. The client and server then can communicate regardless of their origins. Some organisations reported using MCP to deploy AI agents that can navigate complex company internal system from code repositories to data storage with ease. This would be previously accomplished after few months of custom integration work per host and server.

Improvement in engineering efficiency in the organisational level at the same time still requires deeper investigation on security and governance check when deploying AI system to production. A topic of authorisation, logging and accountability are critical concerns for the organisations as AI agent now can read from and write to enterprise ecosystem. Certain security and malicious attacks are common threats for the organisations, and these risks needed robust governance frameworks to prevent any negative implication. This put burden to the organisations but not a main reason to avoid MCP. Same protection in development and deployment is needed as to other main critical infrastructure in the enterprises.

Introduction of MCP to the public eyes delivered significant impacts to society level. Since MCP is an open-source protocol and model-agnostic, it reduces the barrier for smaller team and independent developers in building AI systems that can connect to the live data record. This is very important especially in a part of the world where large engineering team could not be afforded in building many custom integrations. MCP gives bigger goals and aspiration in AI development in a society due to its public-interest decision.

Concerns on accountability in agentic AI become more prominent with MCP. When AI agent is so powerful in taking critical action via MCP's tool calling, the question of who takes the responsibility for that particular action if anything goes wrong has not been clearly answered and laid out. No existing legal or regulatory guidelines has been properly established to close this gap and provide clear direction for MCP governance. Therefore, this issue still represents one of the critical outstanding challenges in AI.

It is worth noting on how MCP tells us the build of foundational AI infrastructure. The donation of MCP by Anthropic to Linux Foundation is supported and backed by other AI players such as OpenAI, Google, Microsoft and Amazon. These supports signal a collective recognition that certain AI architectures are too precious to be owned and handled by a single provider. The governance of MCP created a strong and community-driven infrastructure rather than a profitable proprietary product.

MCP standardises the communication between AI agent and external environment, but it does not do the same between AI agents themselves. The gap exists in which AI deployment nowadays moves from single agent system towards complex multi agent system to accomplish tasks that no longer can be handled by single agent alone. Introduction of Agent2Agent (A2A) protocol by Google complements MCP by addressing the challenges identified in deploying large-scale multi agent system. As MCP standardises how AI agent connects to a data record, file system and external environment, A2A standardises how one AI agent communicate with the other agents. Similar like MCP, A2A uses JSON-RPC over HTTP for communication between client agent and server agent. A2A creates a standard for collaboration between AI agents themselves, provides agent discovery framework and also accommodates secure inter-agent communication. The dynamic between both MCP and A2A suggests the evolution of agentic AI infrastructure and not to be treated as a competition between the two protocols. What is clear is that both protocols were designed to tackle current problems in context understanding and inter-agent operability in AI system.

Conclusion

The arrival of MCP is in the correct time where solution for AI scalability is much needed. LLM's capabilities and its reasoning power grew enormously over the years but still lacked in the sense of utilizing user context stored in a separate location. MCP solved connectivity inefficiency that has been persisted across several years by establishing a universal and standard open protocol between AI application and tool. The breadth and speed of its adoption since the official release illustrated a success story for AI community. MCP showed that some point in AI progression timeline, the advantages of its emerging infrastructure outweighed the competitive benefits of proprietary product. MCP well-designed protocol released at the right moment shaped AI ecosystem and its future.

The benefits of MCP extend beyond AI interoperability and efficiency. For individuals, AI applications become more knowledgeable and powerful as LLM no longer blind to user context and environment. For organisations, MCP significantly reduces the engineering overhead of AI development. Lastly for society impact, MCP proves that big tech companies committed in keeping AI infrastructure open, neutral and community-driven for acceleration of Agentic AI. However, the pros of MCP at the same time do not diminish the concerns over security risks, governance stakes and full accountability in agentic system.

Based on this analysis, AI community should tackle the growing concern of security breach and attack in MCP to the highest priority by developing strong research and robust solutions. Recommendation for organisations would be to adopt MCP as one of the main enterprise infrastructures from the beginning of execution. This refers investing in authentication, consent and logging mechanism before AI applications being deployed for usage. Last but not least, policy makers and regulatory bodies should engage together in making sure accountability practice is reflected in the protocol.

Arguably MCP is still a young protocol, a lot of future work can also be done to strengthen its capabilities. The plan for evolving MCP in its roadmap describes the strategic priorities and what to anticipate from the MCP community. Main priority areas outlined scalability, agent communication, governance maturation and enterprise readiness. In conclusion, MCP makes AI systems connected the world they are meant to and it has proven to be fundamental in the era of agentic AI.

Reference

Anthropic. (2024, November 25). *Introducing the Model Context Protocol*. anthropic.com/news/model-context-protocol

Anthropic. (2025, December 9). *Donating the Model Context Protocol and establishing the Agentic AI Foundation*. anthropic.com/news/donating-the-model-context-protocol

European Commission. (2024). *EU Artificial Intelligence Act*. europa.eu/info/law/better-regulation/have-your-say/initiatives/12527-Artificial-intelligence-ethical-and-legal-requirements

Google. (2025, April 9). *Announcing the Agent2Agent Protocol (A2A)*. Google Developers Blog. developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability

Gupta, D. (2025, December 11). *The complete guide to Model Context Protocol: Enterprise adoption, market trends and implementation strategies*. guptadeepak.com

Linux Foundation. (2025). *Agentic AI Foundation (AAIF)*. linuxfoundation.org

Microsoft. (2025, May 19). *Windows 11 as agentic OS with native MCP support*. Announced at Microsoft Build 2025.

Model Context Protocol. (2025). *Official specification v2025-11-25*. modelcontextprotocol.io/specification/2025-11-25

Model Context Protocol. (2025). *Roadmap - Model Context Protocol.* modelcontextprotocol.io/development/roadmap

OpenCV. (2025, April 4). *A beginner's guide on Model Context Protocol (MCP)*. opencv.org/blog/model-context-protocol

Pento.ai. (2025, December). *A year of MCP: From internal experiment to industry standard*. pento.ai/blog/a-year-of-mcp-2025-review

Raina, A. (2025, December 23). *One year of Model Context Protocol: From experiment to industry standard*. ajeetraina.com

Security Researchers. (2025, April). *MCP security analysis: Prompt injection, tool permissions and lookalike tools*. Cited in Wikipedia — Model Context Protocol article. en.wikipedia.org/wiki/Model_Context_Protocol

Taft, D. K., & Lawson, L. (2025, December 18). *Why the Model Context Protocol won*. The New Stack. thenewstack.io/why-the-model-context-protocol-won

Tran, K-T., Dao, D., Nguyen, M-D., Pham, Q-V., O'Sullivan, B., & Nguyen, H. D. (2025). *From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems*. arXiv. arxiv.org/abs/2505.03864